

Generic and Dynamic Firmware for IO-Box

Selim

April 2026

Contents

1	PROJECT GENERAL CONTEXT AND PRESENTATION OF THE COMPANY	4
1.1	Introduction	4
1.2	Host Organization	4
1.2.1	Presentation of the Host Organization	4
1.2.2	Organization Chart and Company Services	5
1.2.3	Projects and Services of Be Wireless Solutions	5
1.3	Project Specification	8
1.3.1	Problematics	8
1.3.2	General Purpose Input Output Box	8
1.3.3	Methodology: V-Model	9
1.3.4	Objectives	10
1.3.5	Expected Impact	11
1.3.6	Conclusion	11
2	GENERAL CONCEPT AND SYSTEM COMPONENTS	12
2.1	Introduction	12
2.2	Synoptic Diagram of the Proposed Solution	12
2.3	General Idea Presentation and Used Tools Utility	13
2.4	Hardware Tools	15
2.4.1	STM32G474	15
2.5	Software Tools	16
2.5.1	Visual Studio Code (VS Code)	16
2.5.2	IAR Embedded Workbench	17
2.5.3	STM32CubeMX	18
2.5.4	GitLab	18
2.6	Communication Protocols	19
2.6.1	Serial Peripheral Interface (SPI)	19
2.6.2	Controller Area Network (CAN)	20
2.7	Peripherals	21
2.7.1	Analog to Digital Converter (ADC)	21
2.7.2	Timers	22
2.7.3	Pulse Width Modulation (PWM)	23
2.7.4	EEPROM Memory: M95M01-RDW6TP	24
2.8	Sensors	24
2.8.1	ENS210 Sensor	25
2.8.2	OPT3001DNPR Sensor	25

2.8.3	Conclusion	26
3	Realization	28
3.1	introduction	28
3.2	Firmware Architecture	28
3.3	Board Support Package (BSP)	29
3.3.1	CubeMX Project Configuration	30

General Introduction

Embedded systems are computing units integrated into physical devices and designed to carry out specific functions under constrained resources. They are pervasive in domains such as automotive engineering, medical technology, industrial automation, and consumer electronics, where reliability and efficiency are indispensable. Because these systems operate under strict limitations and often in real-time conditions, their conception and implementation represent a demanding and strategically important branch of modern engineering.

Central to the operation of embedded systems is firmware, the dedicated software that directly governs hardware behavior. Firmware ensures that microcontrollers, sensors, and peripherals interact correctly and consistently. It forms the hidden interface between the electronic substrate and the application layer, thereby safeguarding stability, performance, and maintainability. In practice, the effectiveness of an embedded system depends less on the sophistication of its hardware than on the robustness of its firmware.

Equally critical is the dimension of communication. Embedded devices rarely function in isolation; they must exchange information with sensors, actuators, and controllers in order to fulfill their role within larger architectures. Communication protocols establish the rules of this exchange, guaranteeing synchronization, reliability, and interoperability across heterogeneous components. Their proper implementation is what allows embedded systems to evolve into connected, intelligent solutions.

In this context, we introduce our project entitled “**Generic and Dynamic Firmware for IO-Box**”, developed at **Be Wireless Solutions**. The project seeks to design a layered firmware architecture that unifies sensors and multiple communication protocols within a coherent and flexible framework. By emphasizing modularity, resilience, and adaptability, the IO-Box firmware aspires to deliver a precise and scalable solution capable of meeting industrial requirements while maintaining clarity and dependability.

Chapter 1

PROJECT GENERAL CONTEXT AND PRESENTATION OF THE COMPANY

1.1 Introduction

This chapter is organized to provide a clear overview of the academic and industrial framework in which our graduation project was carried out. First, we present the academic institution and the host company that supported the development of this work. Next, we describe the organizational structure and services of the company, along with its main products, in order to highlight the environment in which the project was conceived. Finally, we outline the specifications of the project, including its objectives and the concept of the General Purpose Input Output Box, which represents the core of our study.

1.2 Host Organization

This section presents the hosting organization, Be Wireless Solutions, and its role in supporting innovative technological projects.

1.2.1 Presentation of the Host Organization

Be Wireless Solutions (BWS) is a Tunisian company founded in 2016, recognized as one of the pioneers of Internet of Things (IoT) technologies in the region. In 2017, the company obtained its official IoT license, and by 2019 it was awarded the national start-up label, confirming its innovative positioning in the Tunisian technology ecosystem. From its inception, BWS has focused on designing and manufacturing IoT-related technologies, including smart sensors, intelligent boxes, and integrated platforms that allow clients to monitor and control their systems in real time through web and mobile applications. What distinguishes BWS is its ability to deliver complete solutions that combine hardware, software, and communication layers, while also producing its own devices locally in Tunisia.

The company has achieved remarkable growth in the domestic market, marked by a deployment rate of connected devices that increased by more than 200% compared to competitors. This success has been recognized internationally: in 2020, the American magazine **Startup City** listed BWS among the ten most promising technology start-ups in Africa. In 2021, the company won the award for Best Small and Medium Enterprise (SME) during

the Tunisia Digital Days at Expo 2020 in Dubai. In 2022, BWS reached the finals of the MED'INNOVANT AFRICA competition and was named by *MEA Markets* as the best IoT company in Tunisia.

Beyond awards, BWS has built its reputation on sustainability, innovation, and customer satisfaction. The company operates entirely from Tunisia, with its own production facilities dedicated to IoT devices and solutions. Its expertise covers resource optimization, including reducing fuel and maintenance costs, improving energy efficiency in industrial and agricultural sectors, and enabling smart management of fleets, public lighting, and critical infrastructure. BWS solutions also address environmental protection, monitoring electricity consumption, water usage, cold chain logistics, and remote machine control, providing clients with integrated dashboards, alert systems, and AI-based recommendations to support decision-making.

Thanks to its capacity to deliver reliable and innovative solutions, BWS has expanded its reach beyond Tunisia, establishing collaborations in France, Switzerland, Morocco, Senegal, and Qatar. These international partnerships reinforce its role as a trusted provider of IoT solutions at a global scale. Ultimately, the company's strategic vision is to offer creative and sustainable technologies that preserve resources while enabling organizations to achieve greater efficiency and resilience in a connected world.

(image de logo BWS)

1.2.2 Organization Chart and Company Services

This subsection details the company's departments and services, explaining how each contributes to the overall mission of delivering technological solutions.

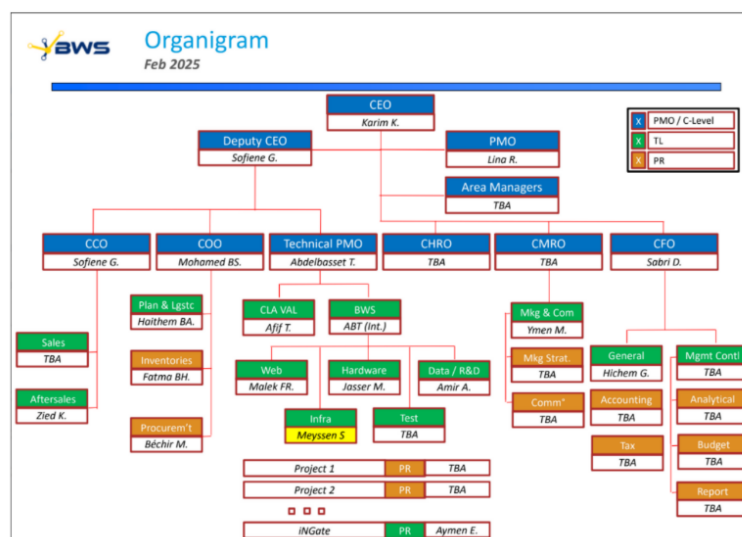


Figure 1.1: Organization Chart

1.2.3 Projects and Services of Be Wireless Solutions

Be Wireless Solutions provides several innovative IoT services and solutions. The main projects and services include:

- **Electricity Consumption Monitoring:** A smart energy management solution that identifies energy-consuming equipment, provides real-time monitoring, and offers AI-based recommendations to reduce costs and improve carbon footprint.



Figure 1.2: Electricity consumption monitoring solution

- **Water Consumption Monitoring:** A system that detects leaks and fraud, enables remote reading of water meters, and helps optimize water usage without requiring major infrastructure changes.

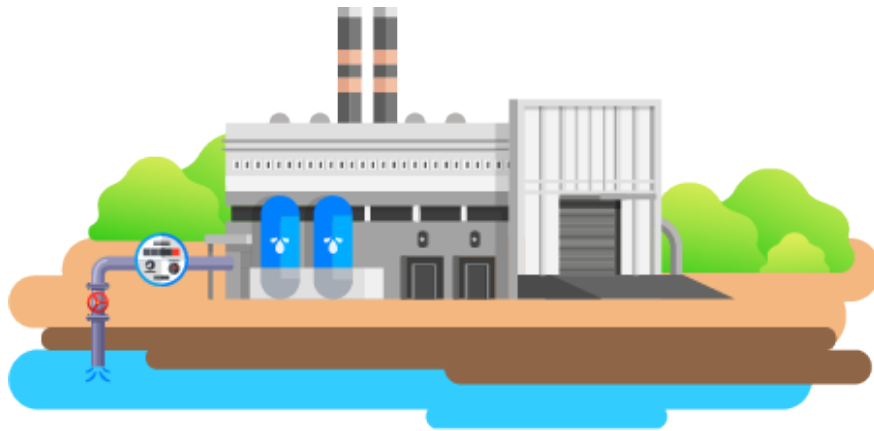


Figure 1.3: Water consumption monitoring solution

- **Fleet and Fuel Management:** An integrated vehicle tracking solution that monitors fuel consumption in real time, supports eco-driving practices, and improves operational efficiency for transport and logistics.



Figure 1.4: Fleet and fuel management solution

- **Remote Control of Equipment:** A cloud-enabled system that allows operators to remotely control machines, detect anomalies, and predict breakdowns, reducing waste and improving reliability in industrial processes.

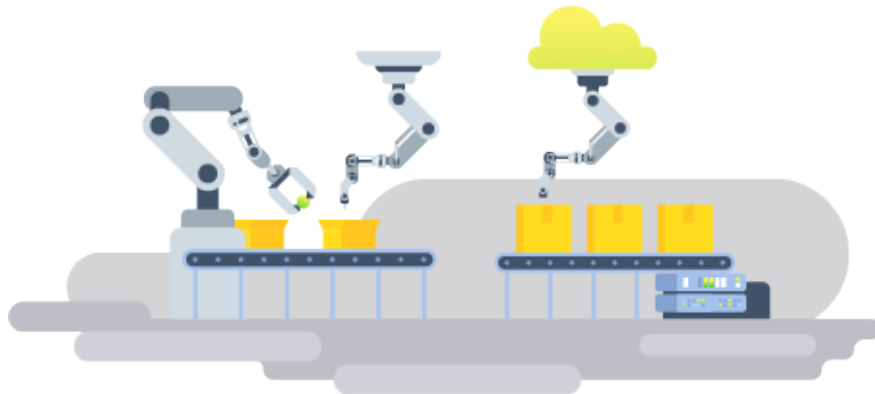


Figure 1.5: Remote control of equipment solution

- **Cold Chain Monitoring:** A wireless monitoring solution that ensures compliance with cold chain requirements in warehouses and transport vehicles, with instant alerts when temperature thresholds are exceeded.



Figure 1.6: Cold chain monitoring solution

1.3 Project Specification

This section defines the specifications of the IO-Box project, outlining its scope and requirements.

1.3.1 Problematics

One of the main challenges encountered by Be Wireless Solutions lies in the complexity and time required to configure and deploy sensors in industrial environments. Each new sensor installation traditionally involves a lengthy process of programming, wiring, and adapting interfaces to ensure compatibility with existing systems. This repetitive and resource-intensive workflow not only slows down deployment but also increases the risk of errors and maintenance difficulties.

To overcome these limitations, BWS has proposed the development of the IO-Box, a general-purpose interfacing platform designed to simplify sensor integration.

1.3.2 General Purpose Input Output Box

A General Purpose Input Output (GPIO) Box is a modular device designed to provide a flexible interface between digital systems and external hardware. It acts as a bridge that allows microcontrollers, processors, or embedded platforms to interact with sensors, actuators, and other peripherals in a standardized way. The GPIO Box typically offers a set of configurable pins that can be programmed either as inputs, to read signals from external devices, or as outputs, to control or drive external components.

In practice, a GPIO Box is widely used in industrial automation, embedded systems, and educational environments because of its versatility. It enables the acquisition of digital and analog signals, supports basic control tasks, and facilitates communication with external modules. By abstracting the complexity of direct hardware connections, the GPIO Box provides a simple and reliable interface for experimentation, prototyping, and system integration.

The main advantages of a General Purpose Input Output Box are its adaptability and ease of use. It can be reconfigured to meet different application requirements, ranging from basic

signal monitoring to more advanced control operations. This makes it a fundamental building block in modern embedded system design, offering engineers and developers a practical tool for connecting and managing diverse hardware components.

1.3.3 Methodology: V-Model

For the development of the IO-Box project, the V-Model (Verification and Validation Model) was adopted as the methodological framework. This structured approach is widely used in software engineering and embedded systems, particularly when a high level of reliability is required. It is based on a rigorous sequential process that organizes development and testing activities into two symmetrical branches forming a "V" (Figure 1.7).

On the descending branch, the process begins with the specification of requirements, followed by functional design and detailed design. These steps ensure that the system's requirements are well defined and that a coherent hardware and software architecture is established. Once the design is completed, the implementation (coding) phase constitutes the core of the technical realization.

The ascending branch is dedicated to verification and validation, including unit tests, integration tests, system tests, and final validation. Each of these phases corresponds to a previous design stage, ensuring that every requirement is verified by appropriate tests. This symmetry between design and testing promotes complete traceability of requirements, which is a key factor in critical projects.

One of the main advantages of the V-Model is the early detection of anomalies during the initial phases of development, significantly reducing the risk of costly errors at the end of the cycle. Moreover, this approach strengthens the quality of the final product by imposing rigorous validation at each level and encouraging clear documentation throughout the project. The V-Model was therefore particularly suited to the context of this project, which required a high degree of reliability and strict management of the development cycle.

The V-Model process applied to this project included the following steps:

- **System Requirements:** Define and document the functional and technical requirements of the IO-Box, including accuracy of measurements, real-time constraints, and reliability of communication.
- **System Design:** Establish the overall architecture of the system, including sensors (ENS210, OPT3001, current sensor), STM32 microcontroller, and communication interfaces.
- **Software Requirements:** Specify the software modules required, such as signal acquisition, filtering, regulation, and communication protocols.
- **Software Implementation:** Develop the firmware for STM32 using IAR Embedded Workbench, ensuring proper configuration of drivers, BSP, and HAL.
- **Software Integration:** Integrate communication, signal processing, and control modules into a coherent application.
- **System Integration:** Deploy the firmware on the STM32 board and connect the system to sensors and actuators for autonomous operation.

- **Validation Tests:** Perform functional and performance tests to verify measurement accuracy, stability of regulation, and reliability of communication with external supervisory systems.

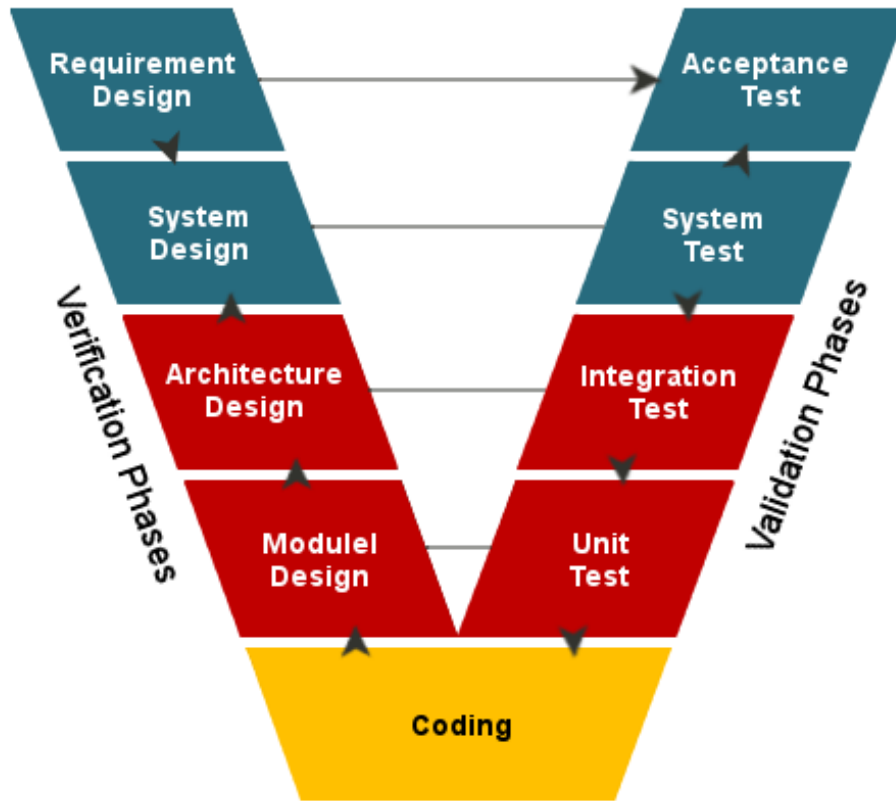


Figure 1.7: Development cycle based on the V-Model

1.3.4 Objectives

The main objective of the IO-Box project is to design and implement a versatile interfacing and signal-processing platform capable of acquiring, processing, and communicating industrial data in real time. More specifically, the system aims to:

- **Signal Acquisition:** Collect heterogeneous analog and digital signals from various sensors and industrial equipment.
- **Interfacing with External Devices:** Communicate with third-party sensors and actuators using multiple industrial protocols.
- **Local Data Processing:** Perform on-board filtering, calculations, and regulation to ensure reliable and efficient operation.
- **Actuator Control:** Drive actuators with precision and robustness, guaranteeing dependable system responses.
- **External Communication:** Exchange data with supervisory systems, applications, or IoT platforms for monitoring and integration.

On the software side, the project emphasizes optimal exploitation of hardware resources through a modular architecture based on hardware abstraction layers (HAL). This includes driver management for peripherals and independence from the underlying hardware.

1.3.5 Expected Impact

The IO-Box is expected to simplify sensor integration, reduce deployment time, and improve operational efficiency in industrial contexts. By offering a standardized interface, it enhances interoperability between devices and ensures scalability for future applications. This impact positions the IO-Box as a practical contribution to modern industrial IoT solutions.

1.3.6 Conclusion

In this chapter, we presented the general framework of the project. We began with an overview of Be Wireless Solutions and highlighted the main problematics related to sensor integration. This led us to define the objectives of the IO-Box project, which aims to provide a modular and efficient platform for signal acquisition, processing, and communication. By clarifying the context and goals, this chapter sets the foundation for the detailed study that follows.

The next chapter will be devoted to the analysis and collection of project requirements. It will specify the functional and technical needs that the IO-Box must address, serving as a basis for the design and implementation phases.

Chapter 2

GENERAL CONCEPT AND SYSTEM COMPONENTS

2.1 Introduction

Building on the general framework and objectives outlined in the previous chapter, this chapter focuses on the technical environment that supports the development of the IO-Box project. It presents the hardware and software components employed, the development tools and programming environment, as well as the communication protocols that enable interaction with sensors, actuators, and external systems. The chapter also introduces the hardware architecture, detailing how the different elements are organized and integrated to form a coherent and functional platform.

By examining these aspects, we establish the practical foundation necessary for the design and implementation of the IO-Box. This technical overview highlights the resources and methodologies that will be used throughout the project, ensuring that the system is both reliable and adaptable to industrial requirements.

2.2 Synoptic Diagram of the Proposed Solution

The proposed solution can be represented by a synoptic diagram that highlights the main functional phases of the system. The process begins with the acquisition of data from the integrated sensors. These raw signals are then subjected to filtering and calibration in order to remove noise and ensure measurement accuracy. The processed data is passed to the regulation stage, where decision-making and control logic are applied. Finally, the regulated information leads to two possible actions: either the transmission of data to external systems (IoT/cloud or monitoring dashboard), or the activation of actuators to directly influence the physical environment.

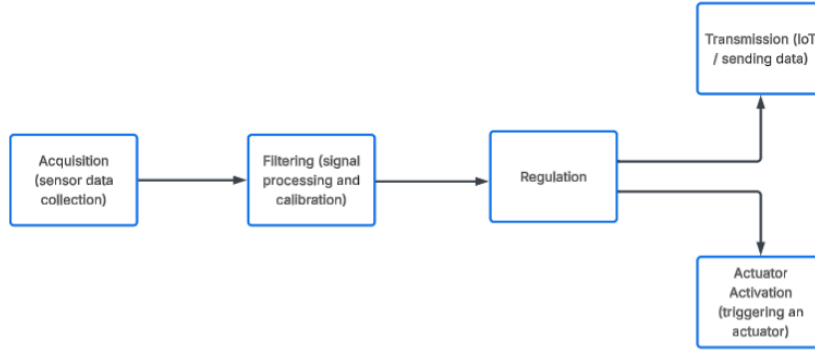


Figure 2.1: Synoptic diagram of the proposed solution showing acquisition, filtering, regulation, and transmission/actuation

2.3 General Idea Presentation and Used Tools Utility

As introduced in the previous chapter, the IO-Box project is conceived as a modular platform designed to simplify sensor integration and signal management in industrial contexts. The general idea is to build a system that acquires inputs, processes them locally, and communicates results to external supervisory platforms. To achieve this, the IO-Box relies on both a layered software architecture and a robust hardware foundation.

Software Layers

The software architecture is organized into four distinct layers:

- **Hardware Abstraction Layer (HAL):** Provides standardized access to peripherals and hardware resources, ensuring portability and independence from the underlying board.
- **Board Support Package (BSP):** Contains drivers and low-level routines specific to the STM32 microcontroller and associated components.
- **Manager Layer:** Supervises system functions, including protocol management, error handling, and data integrity.
- **Application Layer:** Implements the functional logic of the IO-Box, such as signal processing, actuator control, and communication with external supervisory systems.

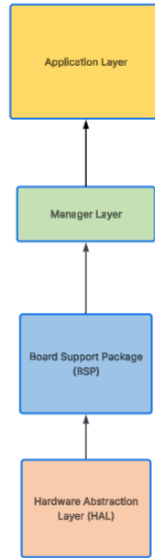


Figure 2.2: Software architecture organized into four distinct layers

Hardware Components

Complementing the software layers, the hardware architecture provides the physical interface with the industrial environment. As shown in the block diagrams provided by the hardware team, the system integrates:

- **Power Supply Architecture:** A multi-stage conversion system that regulates input voltages (12–72V) down to stable levels (48V, 24V, 12V, 5V, and 3.3V) using buck converters and LDO regulators, ensuring reliable energy distribution.
- **Microcontroller Unit (STM32G474):** Serves as the core processing element, coordinating peripherals and executing firmware.
- **Communication Interfaces:** Support for UART, SPI, I²C, CAN FD, LIN, and RS485, enabling interoperability with diverse sensors and industrial devices.
- **Peripheral Modules:** ADCs, DACs, PWM outputs, multiplexers, and transceivers that extend the IO-Box’s connectivity and control capabilities.

Together, these software layers and hardware components form a coherent system that translates the project’s objectives into a practical and scalable solution. This general concept provides the foundation for the detailed technical environment and architecture that will be analyzed in the following sections.

2.4 Hardware Tools

2.4.1 STM32G474

At the heart of the IO-Box hardware architecture lies the STM32G474 microcontroller, a high-performance device from STMicroelectronics. It is based on the ARM Cortex-M4 core with floating-point unit (FPU), making it particularly well-suited for real-time signal processing and control applications.

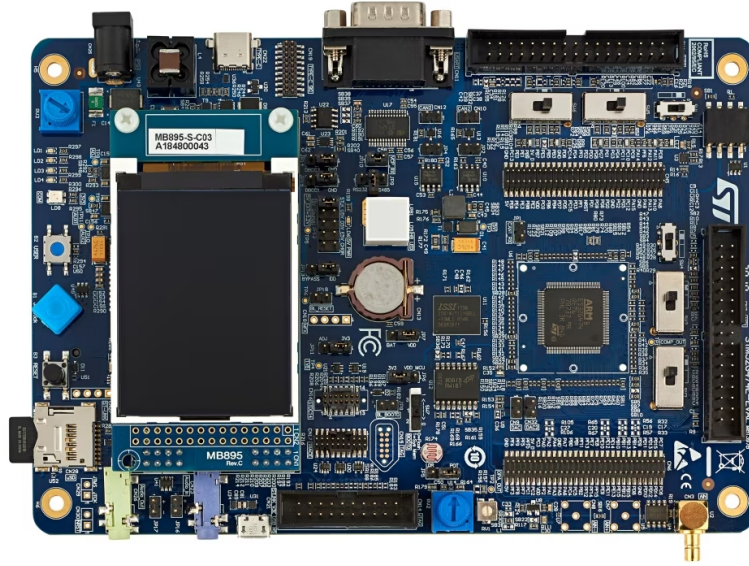


Figure 2.3: STM32G474 Microcontroller (physical package)

The STM32G474 was selected for several reasons:

- **Processing Capabilities:** Equipped with a 170 MHz CPU and DSP instructions, it enables efficient execution of filtering, regulation, and control algorithms.
- **Rich Peripheral Set:** Provides multiple ADC channels, DAC outputs, PWM timers, and communication interfaces (UART, SPI, I²C, CAN FD, LIN, USB), ensuring compatibility with diverse sensors and actuators.
- **Memory Resources:** Integrates up to 128 KB SRAM and 512 KB Flash, allowing the implementation of complex firmware while maintaining flexibility.
- **Analog Performance:** Features high-resolution ADCs and DACs, supporting precise acquisition and generation of industrial signals.
- **Scalability and Ecosystem:** Benefits from the STM32Cube ecosystem, including HAL libraries, BSP drivers, and development tools that simplify firmware design and portability.

Table 2.1: Product Attributes — STM32G474 Microcontroller

Category	Integrated Circuits, Embedded Systems, Microcontrollers
Manufacturer	STMicroelectronics
Series	STM32G4
Packaging	Tray
Component Status	Active
Processor Core	ARM Cortex-M4F
Core Size	32-Bit
Speed	170 MHz
Connectivity	CANbus, I ² C, IrDA, LINbus, QSPI, SPI, UART/USART
Peripherals	DMA, I2S, POR, PWM, WDT, Brown-out detection/reset
Number of I/O	52
Program Memory Size	512 KB Flash
RAM Size	128 KB
Data Converters	ADC: 26 × 12-bit; DAC: 7 × 12-bit
Supply Voltage	1.71 V – 3.6 V
Oscillator Type	Internal
Operating Temperature	–40 °C to +125 °C
Mounting Type	Surface Mount
Package	64-LQFP (10 × 10 mm)
Base Product Number	STM32G474

Within the IO-Box, the STM32G474 acts as the central processing unit, coordinating signal acquisition, local data processing, and actuator control. It also supervises communication with external systems through multiple industrial protocols, making it a key enabler of the project’s objectives.

2.5 Software Tools

2.5.1 Visual Studio Code (VS Code)

Visual Studio Code is employed as a lightweight and versatile integrated development environment (IDE). It provides syntax highlighting, debugging extensions, and Git integration, making it suitable for rapid code editing and version control. Its modularity allows developers to adapt the environment to specific project needs.



Figure 2.4: Visual Studio Code interface

2.5.2 IAR Embedded Workbench

IAR Embedded Workbench is used for compiling and debugging embedded C code on the STM32 platform. It offers advanced optimization features, precise debugging capabilities, and support for hardware breakpoints, which are essential for real-time embedded applications. This tool ensures reliable firmware execution on the target microcontroller.



Figure 2.5: IAR Embedded Workbench IDE

2.5.3 STM32CubeMX

STM32CubeMX is a configuration tool provided by STMicroelectronics. It simplifies the initialization of peripherals, clock trees, and middleware by generating initialization code automatically. This reduces development time and ensures consistency in hardware resource management. It also integrates seamlessly with HAL and BSP layers, aligning with the modular architecture of the IO-Box.



Figure 2.6: STM32CubeMX configuration interface

2.5.4 GitLab

GitLab is employed as a collaborative platform for version control and project management. It enables developers to share code, track changes, and maintain repositories securely. Through GitLab, the team ensures transparency, reproducibility, and efficient collaboration, which are critical for academic and industrial projects alike.



Figure 2.7: GitLab logo

2.6 Communication Protocols

2.6.1 Serial Peripheral Interface (SPI)

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol widely used in embedded systems. It enables high-speed data exchange between a microcontroller and peripheral devices such as sensors, memory modules, or communication transceivers. SPI operates in a master–slave configuration, where the microcontroller typically acts as the master, controlling the clock and data flow.

In the IO-Box project, SPI is employed to ensure reliable and efficient communication with external modules that require fast data transfer. Its simplicity and speed make it suitable for real-time applications, particularly when multiple peripherals must be interfaced with the STM32 microcontroller.

SPI communication is based on a clock signal generated by the master device. Each bit of data is shifted out on the MOSI line while simultaneously another bit is shifted in on the MISO line. This process is synchronized by the clock (SCLK), ensuring deterministic timing. The communication sequence typically follows these steps: the master activates the Chip Select (CS) line of the desired slave, provides the clock signal, transmits data bit-by-bit on MOSI while receiving data on MISO, samples the data according to the configured mode, and finally deactivates the CS line to end the session. This mechanism allows full-duplex communication, meaning data can be transmitted and received simultaneously.

SPI supports four modes of operation (Mode 0 to Mode 3), determined by the configuration of clock polarity (CPOL) and clock phase (CPHA). This flexibility ensures compatibility with a wide range of peripheral devices. Key characteristics of SPI include its high speed (operating at several MHz), simple wiring with four main signals (MOSI, MISO, SCLK, CS), and scalability through multiple slave connections.

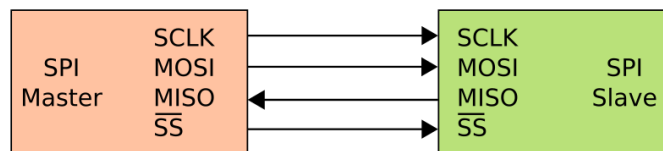


Figure 2.8: Basic SPI communication between master and slave devices

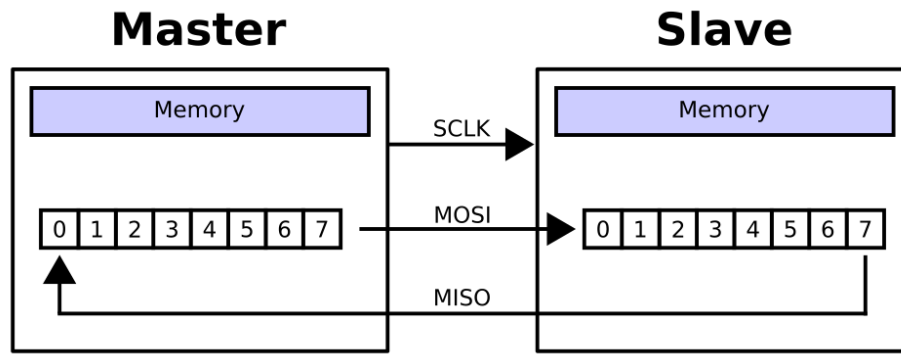


Figure 2.9: Comparison of Big-endian and Little-endian byte ordering in memory

2.6.2 Controller Area Network (CAN)

The Controller Area Network (CAN) is a robust serial communication protocol originally developed for automotive applications, but now widely used in industrial and embedded systems. It enables reliable data exchange between multiple devices without requiring a central host, making it ideal for distributed control systems.

In the IO-Box project, CAN is employed to ensure communication with industrial equipment and sensors that demand high reliability and fault tolerance. Its ability to handle multiple nodes on a single bus makes it particularly suitable for environments where numerous devices must share information in real time.

CAN communication is based on a message-oriented protocol. Instead of addressing specific devices directly, each message carries an identifier that defines its priority and content. This allows multiple nodes to listen simultaneously and decide whether to act on the received data. The arbitration mechanism ensures that when two nodes attempt to transmit at the same time, the message with the highest priority (lowest identifier value) wins, while the other node waits until the bus is free. This guarantees deterministic communication without collisions.

Key characteristics of CAN include:

- **Multi-master capability:** Any node can initiate communication when the bus is idle.
- **Error detection and handling:** Built-in mechanisms such as CRC checks, acknowledgment bits, and error frames ensure data integrity.
- **Priority-based arbitration:** Messages are prioritized by identifiers, ensuring that critical data is transmitted first.
- **Scalability:** Supports up to 112 nodes on a single bus, making it suitable for complex systems.
- **Speed:** Operates up to 1 Mbps in classical CAN, and higher in CAN FD (Flexible Data-Rate), which extends payload size and transmission speed.

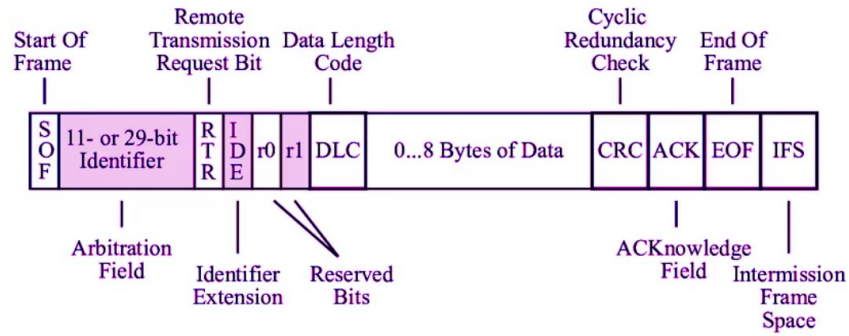


Figure 2.10: Basic CAN frame

In practice, CAN uses two differential lines, CAN_H and CAN_L, to transmit data. This differential signaling improves noise immunity and allows reliable communication even in electrically harsh environments. Each frame consists of fields such as the start of frame, identifier, control bits, data payload, CRC, and end of frame, ensuring structured and error-resistant communication.

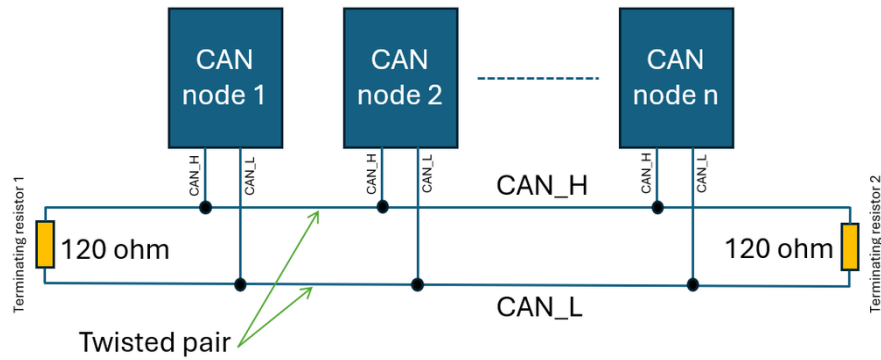


Figure 2.11: Basic CAN bus communication with multiple nodes

2.7 Peripherals

2.7.1 Analog to Digital Converter (ADC)

The Analog-to-Digital Converter (ADC) is a fundamental peripheral in embedded systems, responsible for transforming continuous analog signals into discrete digital values that can be processed by the microcontroller. This conversion is essential for applications where physical phenomena such as temperature, pressure, or current must be measured and interpreted by digital logic.

In the IO-Box project, ADCs are employed to acquire signals from sensors and external modules, enabling precise monitoring and control. The STM32G474 microcontroller integrates multiple high-resolution ADC channels, allowing simultaneous sampling of different inputs. This capability is particularly important in industrial contexts where multiple signals must be captured in real time.

The operation of an ADC follows a well-defined process. The analog input is sampled at regular intervals determined by the sampling frequency. Each sample is then quantized into

a digital value according to the converter's resolution (e.g., 12-bit or 16-bit). The resulting digital word represents the amplitude of the analog signal at that instant. The accuracy of this conversion depends on factors such as resolution, sampling rate, and reference voltage stability.

Key characteristics of ADCs include:

- **Resolution:** Defines the number of discrete levels available.
- **Sampling Rate:** Determines how frequently the analog signal is measured, affecting the fidelity of dynamic signals.
- **Reference Voltage:** Sets the maximum measurable input range, ensuring proper scaling of the digital output.
- **Input Channels:** Multiple channels allow acquisition from several sensors simultaneously.
- **Conversion Modes:** Support for single, continuous, and DMA-driven conversions, enabling flexible integration into firmware.

Within the IO-Box, ADCs are used to capture sensor data such as voltage levels, current measurements, or other analog signals. These values are then processed by the firmware to generate meaningful information, trigger control actions, or communicate results to external supervisory systems.

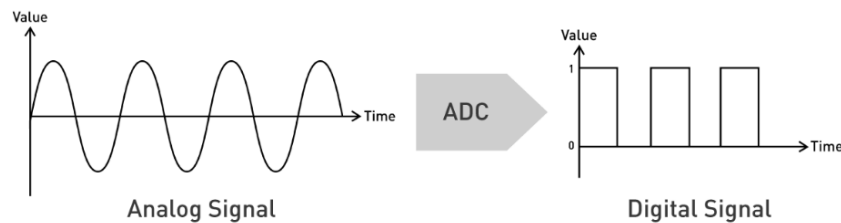


Figure 2.12: Basic ADC conversion from analog input to digital output

2.7.2 Timers

Timers are fundamental peripherals in the STM32G474 microcontroller, designed to provide accurate time base generation, event counting, and signal measurement. They operate by incrementing or decrementing a counter register according to a clock source, and can trigger actions when predefined values are reached.

Key functionalities of timers include:

- **Time Base Generation:** Creating precise delays or periodic events.
- **Input Capture:** Measuring external signal characteristics such as frequency or pulse width.
- **Output Compare:** Generating events when the counter matches a reference value.
- **Synchronization:** Coordinating multiple timers for complex control tasks.

- **Advanced Features:** Dead-time insertion, break inputs, and complementary outputs for safe power electronics control.

Timers are essential for scheduling tasks, measuring sensor signals, and driving peripherals that require precise timing.

2.7.3 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) is a technique used to encode analog information into a digital waveform by varying the duty cycle of a periodic signal. In embedded systems, PWM is widely used for motor control, LED dimming, and power regulation.

PWM signals in the STM32G474 are generated using timers. The timer counts up to a predefined value stored in the auto-reload register (ARR), while the compare register (CCR) defines the switching point of the output. The ratio between the high time and the total period represents the duty cycle. By adjusting ARR and CCR, both frequency and duty cycle can be controlled with high precision.

Key characteristics of PWM include:

- **Frequency:** Determined by prescaler and ARR configuration.
- **Duty Cycle:** Defined by CCR relative to ARR.
- **Resolution:** Limited by timer bit width (typically 16-bit).
- **Complementary Outputs:** Enable safe control of MOSFET drivers in half-bridge or full-bridge circuits.
- **Applications:** Motor drivers, LED dimming, switching converters, and DAC emulation.

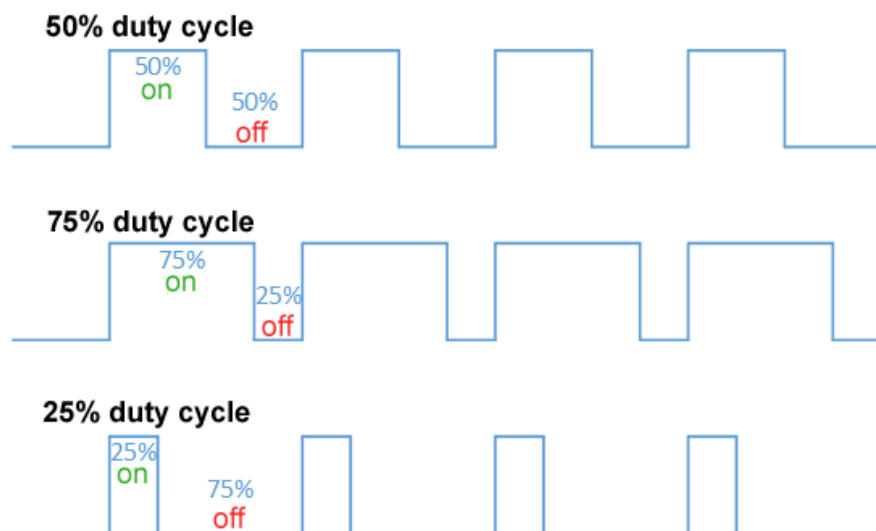


Figure 2.13: Illustration of PWM signals with different duty cycles (25%, 50%, and 75%)

2.7.4 EEPROM Memory: M95M01-RDW6TP

To complement the STM32 microcontroller and ensure reliable data storage, the IO-Box integrates an external EEPROM memory, specifically the M95M01-RMN6TP from STMicroelectronics. This device provides non-volatile storage for configuration parameters, calibration data, and system logs, allowing the platform to retain critical information even after power cycles.

The M95M01-RMN6TP is a 1-Mbit serial EEPROM organized as $131,072 \times 8$ bits. It features a fast access time of 25 ns and communicates via the SPI bus, ensuring efficient integration with the STM32G474. Packaged in an 8-pin SOIC format, it offers a compact solution suitable for embedded applications. Its endurance of up to one million write cycles and data retention of 40 years make it a robust choice for industrial environments where reliability is essential.

Table 2.2: Key Characteristics of M95M01-RDW6TP EEPROM

Feature	Specification
Manufacturer	STMicroelectronics
Type	Serial EEPROM
Capacity	1 Mbit ($131,072 \times 8$ bits)
Interface	SPI
Access Time	25 ns
Package	SOIC-8
Endurance	1,000,000 write cycles
Data Retention	40 years
Operating Voltage	1.8 V – 5.5 V

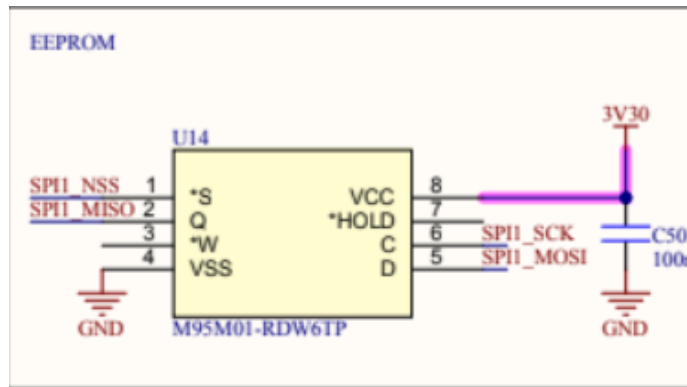


Figure 2.14: M95M01-RDW6TP EEPROM memory chip

2.8 Sensors

In the IO-Box project, several sensors were integrated to enable environmental and physical measurements. Our work focused not only on interfacing these sensors with the STM32G474 microcontroller, but also on configuring their registers, managing communication protocols, and performing calibration to ensure accurate data acquisition. Each sensor was studied in detail, and its characteristics were documented to highlight its role within the system.

2.8.1 ENS210 Sensor

The ENS210 is a digital sensor from ScioSense that integrates both temperature and humidity measurement in a compact package. It communicates via an I²C interface, making it easy to integrate with microcontrollers such as the STM32G474. Its high accuracy and low power consumption make it suitable for environmental monitoring in embedded systems.

Table 2.3: Key Characteristics of ENS210 Sensor

Feature	Specification
Manufacturer	ScioSense
Type	Temperature + Humidity Sensor
Interface	I ² C (16-bit output registers)
Supply Voltage	1.71 V – 3.6 V
Temperature Range	–40 °C to +100 °C
Temperature Accuracy	±0.15 °C
Humidity Range	0 – 100 % RH
Humidity Accuracy	±2 % RH
Package	4-QFN (2 × 2 mm)

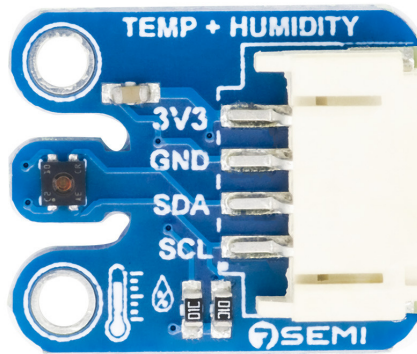


Figure 2.15: ENS210 digital temperature and humidity sensor

2.8.2 OPT3001DNPR Sensor

The OPT3001DNPR is a digital ambient light sensor from Texas Instruments. It is designed to measure the intensity of visible light and provide results in lux units, making it ideal for applications such as automatic brightness control, display backlight adjustment, and environmental monitoring. The sensor communicates via the I²C protocol and features built-in calibration and linear response across a wide dynamic range.

In the IO-Box project, the OPT3001DNPR was integrated to provide accurate light measurements. Work was carried out on its internal registers to configure measurement modes, thresholds, and calibration routines, ensuring reliable data acquisition for real-time applications.

Table 2.4: Key Characteristics of OPT3001DNPR Sensor

Feature	Specification
Manufacturer	Texas Instruments
Type	Ambient Light Sensor
Interface	I ² C
Supply Voltage	1.6 V – 3.6 V
Measurement Range	0.01 lux – 83,000 lux
Output Format	16-bit digital result in lux
Accuracy	±10% typical
Operating Temperature	–40 °C to +85 °C
Package	6-Pin WSON (2 × 2 mm)



Figure 2.16: OPT3001DNPR ambient light sensor

2.8.3 Conclusion

In this chapter, we presented the general concept and system components of the IO-Box project. We began with the synoptic diagram, which outlined the main functional phases of acquisition, filtering, regulation, and transmission/actuation. We then introduced the software architecture, organized into four distinct layers (HAL, BSP, Manager, and Application), and complemented this with the hardware architecture, highlighting the STM32G474 microcontroller, power supply design, communication interfaces, and peripheral modules. Finally, we described the development tools and protocols that support the implementation of the system.

By establishing this technical foundation, Chapter 3 has clarified the resources and methodologies that will be employed in the realization phase. The next chapter will be dedicated to the practical implementation of the IO-Box, detailing the firmware design, register configuration, BSP development, and integration of sensors and actuators into a coherent and functional platform.

Chapter 3

Realization

3.1 introduction

This chapter is dedicated to the practical realization of the IO-Box project, focusing on the firmware development carried out across the different software layers. While the previous chapters introduced the system concept, architecture, and components, the present chapter illustrates the concrete implementation work that transformed the design into a functional prototype.

The realization process is organized around three main layers:

- **Board Support Package (BSP):** Development of low-level drivers and routines specific to the STM32G474 microcontroller and its peripherals, ensuring proper initialization and hardware control.
- **Manager Layer:** Implementation of supervisory modules responsible for protocol handling, error detection, and data integrity, providing a reliable interface between BSP and application logic.
- **Application Layer:** Integration of functional logic, including signal acquisition, filtering, regulation, actuator control, and communication with external supervisory systems.

By detailing the coding and integration performed in each of these layers, this chapter demonstrates how the IO-Box firmware was realized in practice, highlighting the modular approach and ensuring traceability from design specifications to executable code.

3.2 Firmware Architecture

The firmware of the IO-Box was designed according to a layered architecture, ensuring modularity, portability, and clear separation of responsibilities. Figure 3.1 illustrates the overall structure, which is organized into six main components:

- **Application layer:** Peripheral calls and business logic. Handles high-level commands, events, and callbacks.
- **Manager layer:** Supervises peripherals and protocols (EEPROM, UART, RS485, SPI/I2C sensors, DAC, IO logic, CAN-FD). Provides state machines, buffering, coordination, and

bus/channel mapping.

- **BSP layer:** Implements low-level drivers (UART, SPI, I2C, ADC, PWM, CAN, USB, DAC, etc.) as HAL wrappers, exposing standardized APIs to higher layers.
- **Utilities:** Shared helpers such as `Utils.h`, board configuration, validation macros, and debug/logging.
- **Callbacks & ISR:** HAL interrupts, callback functions, and event posting.
- **HAL layer:** Vendor-provided hardware abstraction library used internally by BSP.

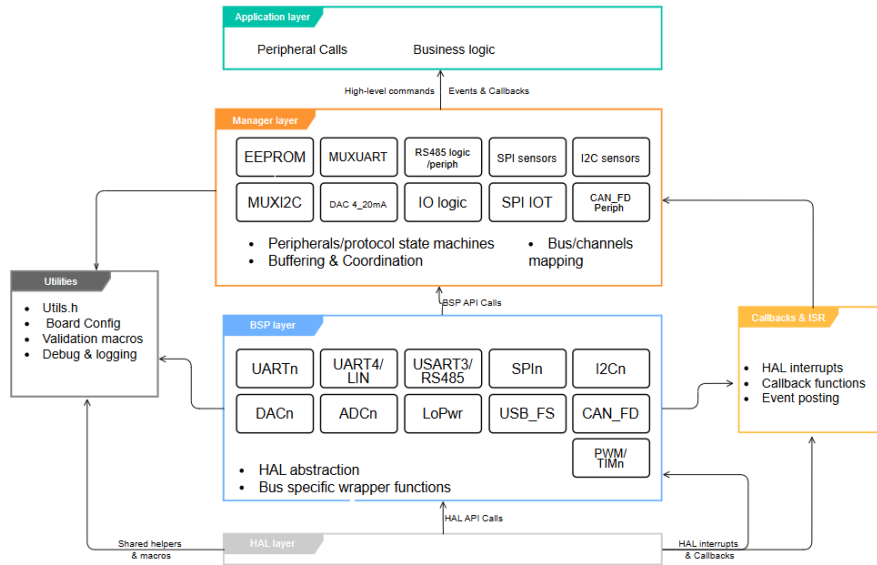


Figure 3.1: Layered firmware architecture of the IO-Box

This architecture provides a clear hierarchy: the BSP abstracts hardware access, the Manager coordinates protocols and resources, and the Application layer implements the functional logic. Utilities and callbacks support all layers, while HAL remains the base hardware abstraction.

3.3 Board Support Package (BSP)

The Board Support Package (BSP) represents the lowest software layer of the IO-Box firmware. It provides the essential drivers and initialization routines that allow the STM32G474 microcontroller to interact with its peripherals in a standardized way. By abstracting hardware details, the BSP ensures that higher layers of the firmware can access resources such as timers, ADCs, communication interfaces, and memory modules without depending directly on register-level configurations.

In the context of this project, the BSP was developed to replace generic HAL usage with custom, instance-specific drivers. This approach guarantees modularity, portability, and precise control over hardware behavior, forming the foundation upon which the Manager and Application layers are built.

3.3.1 CubeMX Project Configuration

This subsection presents the global configuration of the STM32G474 microcontroller performed in CubeMX, including pin assignments and clock setup. Figure 3.2 illustrates the complete pinout used in the IO-Box firmware project.

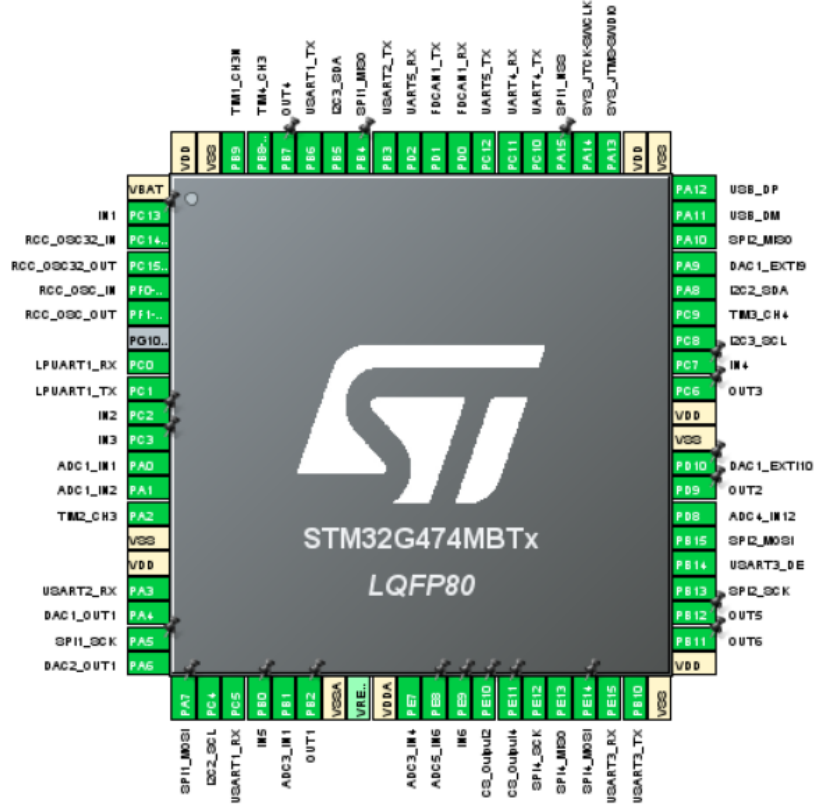


Figure 3.2: STM32G474 CubeMX pinout configuration

SPI2 BSP Implementation

The SPI2 BSP was implemented to enable communication with the EEPROM memory module integrated by the hardware team. Configuration parameters such as baud rate, polarity, phase, and data size were defined in `BSP_SPI2.h` using macros. Figure 3.3 shows the CubeMX configuration interface used to initialize SPI2 in Mode 0 with a prescaler of 16.

```

/**
 * @brief SPI2 PARAMETERS (sensor-friendly defaults)
 */
#define SPI2_BAUDRATE      SPI_BAUDRATEPRESCALER_16    // ~4 MHz if APB1=64 MHz
#define SPI2_DATASIZE      SPI_DATASIZE_8BIT
#define SPI2_POLARITY      SPI_POLARITY_LOW             // Mode 0
#define SPI2_PHASE         SPI_PHASE_1EDGE
#define SPI2_FIRSTBIT      SPI_FIRSTBIT_MSB
#define SPI2_MODE          SPI_MODE_MASTER
#define SPI2_DIRECTION     SPI_DIRECTION_2LINES
#define SPI2_NSS_MODE      SPI_NSS_SOFT                /* sensors use GPIO CS */
#define SPI2_TIMODE        SPI_TIMODE_DISABLE
#define SPI2_CRC_ENABLE    SPI_CRCCALCULATION_DISABLE
#define SPI2_CRC_POLY      7

```

Figure 3.3: example of SPI2 configuration in SPI2BSP.h

The API follows the same structure applied to all BSP drivers: - **Initialization**: BSP_SPI2_init, BSP_SPI2_pins_init, BSP_SPI2_dma_init - **Basic I/O**: BSP_SPI2_read, BSP_SPI2_write, BSP_SPI2_transfer - **Non-blocking I/O**: BSP_SPI2_read_IT, BSP_SPI2_write_DMA, etc. - **Control**: Chip Select management, reset/deinit, buffer flush - **State**: BSP_SPI2_getstate for monitoring peripheral status

This structured design ensures that SPI2 can reliably serve the EEPROM module while remaining reusable for other peripherals if needed. The detailed EEPROM management logic will be presented in the Manager section.

```

/**
 * @brief Initialization
 */
BspStatus_t BSP_SPI2_init(void);
BspStatus_t BSP_SPI2_pins_init(void);
BspStatus_t BSP_SPI2_dma_init(void);

/**
 * @brief Deinit / Reset
 */
BspStatus_t BSP_SPI2_deinit(void);
BspStatus_t BSP_SPI2_reset(void);

/**
 * @brief Basic Read/Write
 */
BspStatus_t BSP_SPI2_read(uint8_t *rx_buffer, uint16_t size);
BspStatus_t BSP_SPI2_write(uint8_t *tx_buffer, uint16_t size);
BspStatus_t BSP_SPI2_transfer(uint8_t *tx_buffer, uint8_t *rx_buffer, uint16_t size);

/**
 * @brief Interrupt / DMA
 */
BspStatus_t BSP_SPI2_read_IT(uint8_t *rx_buffer, uint16_t size);
BspStatus_t BSP_SPI2_write_IT(uint8_t *tx_buffer, uint16_t size);
BspStatus_t BSP_SPI2_read_DMA(uint8_t *rx_buffer, uint16_t size);
BspStatus_t BSP_SPI2_write_DMA(uint8_t *tx_buffer, uint16_t size);
BspStatus_t BSP_SPI2_transfer_DMA(uint8_t *tx_buffer, uint8_t *rx_buffer, uint16_t size);

/**
 * @brief Abort / State / Flush / Reconfig
 */
BspStatus_t BSP_SPI2_abort(void);
HAL_SPI_StateTypeDef BSP_SPI2_getstate(void);

```

Figure 3.4: SPI2 API's in SPI2BSP.h